KUBERNETES

CHEAT SHEETS

A Complete Reference for Running Apps on Kubernetes

Architecture · kubectl · Manifests & Labels · Pods · Deployments · StatefulSets · Jobs · Probes · Services · Gateway API · Network Policies · ConfigMaps & Secrets · Storage · Scheduling · Autoscaling · RBAC · Pod Security · Debugging · Helm & Kustomize

25 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & Architecture
02 Local Clusters & Installing kubectl
03 kubeconfig, Contexts & Namespaces

KUBECTL ESSENTIALS

04 Inspecting Resources & Output Formats
05 Imperative vs Declarative
06 Manifests, Labels & Selectors

WORKLOADS

07 Pods
08 Deployments & Rollouts
09 StatefulSets & DaemonSets
10 Jobs & CronJobs
11 Health Probes & Pod Lifecycle

NETWORKING

12 Services & DNS
13 Ingress & Gateway API
14 Network Policies

CONFIGURATION & STORAGE

15 ConfigMaps & Secrets
16 Volumes & Persistent Storage

RESOURCES, SCHEDULING & SCALING

17 Requests, Limits & Quotas
18 Scheduling & Node Placement
19 Autoscaling & Disruption Budgets

SECURITY

20 RBAC & ServiceAccounts
21 Pod Security

OPERATIONS & TOOLING

22 Debugging & Troubleshooting
23 Helm & Kustomize

QUICK REFERENCE

24 Quick Reference — kubectl Commands
25 Quick Reference — Manifests & Troubleshooting

K8S

Table of Contents

25 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 03

- 01 Introduction & Archite…
- 02 Local Clusters & Install…
- 03 kubeconfig, Contexts …

KUBECTL ESSENTIALS 04 – 06

- 04 Inspecting Resources …
- 05 Imperative vs Declarati…
- 06 Manifests, Labels & Sel…

WORKLOADS 07 – 11

- 07 Pods
- 08 Deployments & Rollouts
- 09 StatefulSets & Daemo…
- 10 Jobs & CronJobs
- 11 Health Probes & Pod Li…

NETWORKING 12 – 14

- 12 Services & DNS
- 13 Ingress & Gateway API
- 14 Network Policies

CONFIGURATION & STORAGE 15 – 16

- 15 ConfigMaps & Secrets
- 16 Volumes & Persistent …

RESOURCES, SCHEDULING & SCALING 17 – 19

- 17 Requests, Limits & Qu…
- 18 Scheduling & Node Pla…
- 19 Autoscaling & Disrupti…

SECURITY 20 – 21

- 20 RBAC & ServiceAccou…
- 21 Pod Security

OPERATIONS & TOOLING 22 – 23

- 22 Debugging & Troubles…
- 23 Helm & Kustomize

QUICK REFERENCE 24 – 25

- 24 Quick Reference — ku…
- 25 Quick Reference — Ma…

Sheet 01

Introduction & Architecture

control plane · kube-apiserver · etcd · scheduler · kubelet · kube-proxy · reconcile loop

WHAT KUBERNETES IS

Kubernetes (K8s) is an open-source **container orchestrator**: it places containers on a pool of machines, restarts them when they fail, scales them and gives them stable networking. You **declare the desired state** in YAML; **controllers** keep nudging the actual state toward it, forever. That loop (*observe → diff → act*) explains everything else in this set.

GOOD TO KNOW

- Started at Google, v1.0 in 2015, now a CNCF project; ~3 releases a year
- Nodes run `containerd` or CRI-O, not Docker Engine; Docker-built images still work
- `kubectl` only writes objects to the API; controllers do the work
- Everything is an **object** in the API: Pods, Services, even Nodes

WATCH THE RECONCILE LOOP

```
kubectl create deployment hello --image=nginx:1.27
kubectl scale deployment hello --replicas=3   # desired: 3 pods
kubectl get pods                    # 3 x hello-6d8f9c7b5-…  Running
kubectl delete pod hello-6d8f9c7b5-x7k2p      # "crash" one
kubectl get pods                    # still 3: a new one replaced it
kubectl expose deployment hello --port=80     # stable Service IP
kubectl get nodes -o wide           # machines in the cluster
kubectl get pods -n kube-system     # coredns, kube-proxy, etcd…
kubectl get --raw='/readyz?verbose' # control-plane health
kubectl delete deployment,svc hello # clean up
```

CODE EXPLAINED

- → `create deployment` — stores an object; controllers create the Pods
- → `delete pod` — actual (2) ≠ desired (3), so a replacement appears
- → `expose` — a Service gives pods one stable IP + DNS name
- → `kube-system` — namespace for cluster components
- → `/readyz` — replaces deprecated `get componentstatuses`
- → `get nodes -o wide` — node IPs, OS, kernel, runtime version

PRODUCTION CHECK

- Use a managed control plane unless you must run your own
- Everything is reconciled — fix the spec, not the pod

IN DOCKER / COMPOSE

- Docker runs containers; K8s keeps a desired state running
- Swarm is Docker's own (smaller) orchestrator

CONTROL PLANE

COMPONENT	JOB
`kube-apiserver`	Front door; every read and write goes through it
`etcd`	Key-value store holding all cluster state
`kube-scheduler`	Picks a node for each new Pod
`controller-manager`	Runs built-in controllers (Deployment, Job, Node…)
`cloud-controller`	Cloud load balancers, routes, node lifecycle
`etcd quorum`	3 or 5 members; losing a majority stops writes

EVERY WORKER NODE

COMPONENT	JOB
`kubelet`	Starts the node's Pods, reports their status
`containerd / CRI-O`	Container runtime that runs the containers
`kube-proxy`	Programs Service routing (iptables / IPVS / nftables)
`CNI plugin`	Pod networking: Calico, Cilium, Flannel
`CoreDNS`	Cluster DNS add-on, runs as pods
`Pod`	Smallest unit: 1+ containers sharing IP + volumes

⚠ COMMON MISTAKES

- Bare Pods for apps — nothing recreates them if the node dies; use a Deployment
- Assuming `apply` succeeded = app running; controllers act afterward
- Fixing things on nodes by hand — controllers overwrite drift
- Expecting `docker ps` on nodes — use `crictl ps`

✓ TIPS & BEST PRACTICES

- Read the Events in `kubectl describe` first; they explain most failures
- Change the spec and let controllers converge; don't script steps
- Learn on a throwaway local cluster (Sheet 02)
- `kubectl api-resources` lists every kind the cluster knows

QUICK SUMMARY

- `apiserver` — the only door; state lives in `etcd`
- `scheduler` — placement; `controllers` — reconcile
- `kubelet` — runs pods on each node
- `Pod` — smallest unit, usually owned by a controller

Local Clusters & Installing kubectl

GETTING A CLUSTER

You need the **kubectl** CLI and a **cluster** to point it at. For learning and CI, run a disposable cluster inside Docker with **kind**, **minikube** or **k3d**; each starts in about a minute. In production most teams use a **managed** control plane (EKS, GKE, AKS) and only run workloads and nodes.

NOTES

- Local tools add a context to `~/.kube/config` automatically
- Locally built images are invisible to the cluster until loaded or pushed
- Docker Desktop has a one-click single-node cluster
- kind names contexts `kind-<name>`; minikube's is `minikube`

SECURITY CHECK

- kubeconfig files are credentials
- Short-lived tokens via the cloud provider CLI

INSTALL & CREATE

```
brew install kubectl                          # macOS
winget install -e --id Kubernetes.kubectl     # Windows
sudo snap install kubectl --classic           # Linux (or dl.k8s.io binary)
kubectl version --client                      # Client Version: v1.3x.x
kind create cluster --name dev                # 1 node in Docker
kind create cluster --name dev --config kind.yaml   # multi-node
kind load docker-image myapp:dev --name dev         # copy image in
minikube start --driver=docker --cpus=4 --memory=8g
minikube addons enable metrics-server         # built-in add-ons
k3d cluster create dev --agents 2             # k3s + 2 workers
kind delete cluster --name dev                # gone in seconds
```

CODE EXPLAINED

- → `version` — without `--client` also shows the server
- → `kind.yaml` — `nodes:` list with `role: control-plane` / `worker`
- → `kind load` — skip the registry for dev images
- → `--agents` — k3d's name for worker nodes
- → `minikube addons list` — ingress, dashboard, registry…
- → `delete cluster` — also removes its kubeconfig context

PRODUCTION CHECK

- Match `kubectl` within one minor version of the server
- Local clusters for dev, managed for prod

IN DOCKER / COMPOSE

- Docker Desktop includes a single-node cluster
- `docker` CLI ↔ `kubectl`

LOCAL OPTIONS

TOOL	BEST FOR
kind	CI, multi-node tests, upstream K8s
minikube	Learning; easy add-ons
k3d	Fast k3s with built-in LB
Docker Desktop	Zero setup on Mac/Windows
k3s	Edge, Pi, small Linux VMs

MANAGED: FETCH CREDENTIALS

SERVICE	COMMAND
EKS	aws eks update-kubeconfig --name c1
GKE	gcloud container clusters get-credentials c1
AKS	az aks get-credentials -g rg -n c1
Create	eksctl · gcloud … create-auto · az aks create

VERSION SKEW

PAIR	ALLOWED
kubectl	±1 minor of API
kubelet	≤ 3 minors older
upgrades	1 minor at a time

⚠ COMMON MISTAKES

- `ErrImagePull` on a local image — `kind load` it or push it
- `:latest` dev tag forces `Always` pull, ignoring the loaded image
- kubectl 2+ minors from the server — odd errors, missing flags
- Leaving local clusters running — they keep eating CPU and RAM

✓ TIPS & BEST PRACTICES

- Pin the node image (`--image kindest/node:v1.3x.y`) to match prod
- Recreate a broken local cluster instead of repairing it
- Manage kubectl versions with `mise` or `asdf`
- Autopilot (GKE) and Fargate (EKS) hide nodes entirely

QUICK SUMMARY

- `kubectl` — brew / winget / snap
- `kind · k3d · minikube` — local clusters
- `get-credentials` — managed clusters
- `skew` — kubectl within one minor of the API

kubeconfig, Contexts & Namespaces

WHERE KUBECTL POINTS

kubeconfig (`~/.kube/config`) holds **clusters** (API URL + CA), **users** (credentials) and **contexts** (cluster + user + default namespace). The `current-context` decides where every command goes. **Namespaces** split one cluster into named areas; names only need to be unique within one.

NOTES

- Without `-n`, commands use the context's namespace, else `default`
- Nodes, PVs and Namespaces are **cluster-scoped**
- kubeconfig holds credentials: `chmod 600`
- `KUBECONFIG` can list several files; kubectl merges them

SECURITY CHECK

- One kubeconfig with prod + dev = easy mistakes
- Separate files or read-only prod contexts

CONTEXTS & NAMESPACES

```
kubectl config get-contexts            # * marks the current one
kubectl config current-context         # kind-dev
kubectl config use-context prod-east   # switch cluster + user
kubectl config set-context --current --namespace=shop
kubectl config view --minify           # active context only
kubectl config delete-context old-lab
export KUBECONFIG=~/.kube/config:~/.kube/eks.yaml   # merge
kubectl config view --flatten > ~/.kube/merged      # one file
kubectl --context prod-east -n shop get pods        # one-off
kubectl create namespace shop
kubectl delete namespace shop          # deletes EVERYTHING in it
```

CODE EXPLAINED

- → `use-context` — rewrites the file; affects every terminal
- → `--current --namespace` — default ns for this context
- → `KUBECONFIG` — `:` separator (`;` on Windows)
- → `--flatten` — inlines certs so the file stands alone
- → `--context` — safest choice in scripts
- → `delete namespace` — waits for every object inside to finalize
- → `view` — redacts secrets; add `--raw` to show them

PRODUCTION CHECK

- Namespaces per team / env; never work in `default`
- Show the current context in your prompt

IN DOCKER / COMPOSE

- kubeconfig contexts ↔ `docker context`
- Namespaces ↔ separate Compose projects (`-p`)

KUBECONFIG KEYS

KEY	HOLDS
`clusters`	`server` URL + CA data
`users`	Token, cert or `exec` plugin
`contexts`	cluster + user + namespace
`current-context`	Active context name
`preferences`	Rarely used; ignore
`exec plugins`	aws, gke-gcloud-auth-plugin, kubelogin

BUILT-IN NAMESPACES

NAMESPACE	PURPOSE
`default`	Used when none given
`kube-system`	Cluster components
`kube-public`	Readable by everyone
`kube-node-lease`	Node heartbeats
`your-app`	One per app/team/env
`cluster-scoped`	Node, PV, Namespace, ClusterRole

KUBECTX / KUBENS

COMMAND	DOES
`kubectx`	List contexts
`kubectx prod`	Switch context
`kubectx -`	Previous context
`kubens shop`	Set namespace
`kubens -`	Previous namespace
`kubectx -d old`	Delete a context

⚠ COMMON MISTAKES

- Deleting in the wrong cluster — check `current-context` first
- `delete namespace` as cleanup — removes PVCs and all
- Treating namespaces as network isolation — they aren't (Sheet 14)
- Committing kubeconfig to Git — it often holds long-lived tokens

✓ TIPS & BEST PRACTICES

- Show context + namespace in your prompt (kube-ps1, starship)
- Name contexts by environment: `dev`, `prod-east`
- Pass `--context` explicitly in CI scripts
- One namespace per app or team per environment

QUICK SUMMARY

- `get-contexts / use-context` — list / switch
- `set-context --current --namespace`
- `-n` one ns · `-A` all namespaces
- `KUBECONFIG=a:b` — merge several files

Sheet 04

Inspecting Resources & Output Formats

get · describe · -o wide / yaml / json · jsonpath · custom-columns · explain · api-resources

READING THE CLUSTER

`kubectl get` lists objects as a table; `-o` switches format, from extra columns to the full live YAML. `kubectl describe` gives a readable summary plus recent **Events**, where most errors show up. `explain` and `api-resources` document the API straight from your cluster.

NOTES

- `get pod web-0` and `get pod/web-0` both work
- List several kinds at once: `get deploy,svc,pods`
- `get all` skips ConfigMaps, Secrets, Ingress, PVCs
- Live YAML includes `status` and server-set fields

INSPECT

```
kubectl get pods -A                      # all namespaces
kubectl get pods -o wide                 # + IP and NODE
kubectl get deploy web -o yaml           # full live object
kubectl get pods -l app=web --show-labels
kubectl get pods --field-selector status.phase!=Running
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pods -w                      # watch changes
kubectl get pod web-0 -o jsonpath='{.status.podIP}'   # 10.244.1.7
kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName
kubectl describe pod web-0               # details + Events
kubectl explain deploy.spec.strategy     # field docs
kubectl api-resources --namespaced=false
```

CODE EXPLAINED

- → `--field-selector` — server-side filter on a few fields
- → `-w` — stream updates until Ctrl-C
- → `jsonpath` — use single quotes in the shell
- → `explain` — types and docs for any field path
- → `api-resources` — every kind and its short name

PRODUCTION CHECK

- `-o yaml` to see what's really applied
- `kubectl explain` instead of guessing fields

IN DOCKER / COMPOSE

- `get` ↔ `docker ps` · `describe` ↔ `docker inspect`
- `--format` ↔ `-o jsonpath`

-O FORMATS

FLAG	GIVES
`-o wide`	Extra columns
`-o yaml|json`	Full object
`-o name`	`kind/name` lines
`-o jsonpath=`	Picked fields
`-o custom-columns=`	Your own table
`-o go-template=`	Go template

SHORT NAMES

SHORT	KIND
`po · deploy · rs`	Pod · Deployment · ReplicaSet
`sts · ds · cj`	StatefulSet · DaemonSet · CronJob
`svc · ing · cm`	Service · Ingress · ConfigMap
`pv · pvc · sa`	PV · PVC · ServiceAccount
`no · ns · hpa`	Node · Namespace · HPA
`sc · ep · secret`	StorageClass · Endpoints · Secret

HANDY FLAGS

FLAG	DOES
`-l k=v`	Label filter
`-L env`	Label as column
`--no-headers`	Drop header
`-v=6`	Show API calls
`{range}`	jsonpath loop
`--show-kind`	Prefix rows with kind

⚠ COMMON MISTAKES

- Trusting `get all` — config, storage and CRDs are missing
- Double-quoted jsonpath in bash — `$` and braces get mangled
- Committing `-o yaml` output — status and uid make noisy diffs
- Skipping the Events at the bottom of `describe` — they name the problem

✓ TIPS & BEST PRACTICES

- `get events --sort-by=.lastTimestamp` for a timeline
- `-o name | xargs` for bulk actions
- `alias k=kubectl` + shell completion
- krew plugin `kubectl neat` strips server fields from YAML

QUICK SUMMARY

- `get` — table; `-o` for more
- `describe` — summary + Events
- `explain` — the API documents itself
- `jsonpath / custom-columns` — script-friendly output

Sheet 05

Imperative vs Declarative

create · run · expose · apply -f · diff · --dry-run=client -o yaml · edit / patch · --server-side

TWO WAYS TO CHANGE A CLUSTER

Imperative commands (`run`, `create`, `scale`, `set`) say *do this now*: quick, but unrecorded. **Declarative** management keeps manifests in Git and runs `kubectl apply`, which creates or updates objects to match the files. Best of both: generate YAML with `--dry-run=client -o yaml`, then `apply` it.

NOTES

- `apply` is idempotent; a second run changes nothing
- `create -f` fails if the object exists
- `edit`, `patch`, `scale` drift from Git
- Server-side apply records a **field manager** per field
- `replace --force` deletes and recreates (immutable fields)

SECURITY CHECK

- `kubectl edit` in prod leaves no history
- Change manifests in git, then apply

IMPERATIVE → DECLARATIVE

```
kubectl run tmp --image=busybox:1.36 --rm -it --restart=Never -- sh
kubectl create deployment web --image=nginx:1.27 --replicas=2
kubectl expose deployment web --port=80 --target-port=8080
kubectl set image deployment/web nginx=nginx:1.28
kubectl create deploy web --image=nginx:1.27 --dry-run=client -o yaml > web.yaml
kubectl apply -f web.yaml               # create or update
kubectl apply -f k8s/ -R                # whole folder
kubectl diff -f web.yaml                # preview changes
kubectl apply -f web.yaml --server-side --field-manager=ci
kubectl edit deployment web             # live object in $EDITOR
kubectl patch deployment web -p '{"spec":{"replicas":3}}'
kubectl delete -f web.yaml              # remove what it defines
```

CODE EXPLAINED

- → `--rm -it --restart=Never` — interactive, deleted on exit
- → `--dry-run=client -o yaml` — prints YAML, creates nothing
- → `diff` — exits 1 when there are changes; a CI gate
- → `--server-side` — API server merges and tracks field owners
- → `patch -p` — strategic merge by default
- → `edit` — saved on exit; invalid YAML reopens the editor
- → `delete -f` — deletes only objects named in the file

PRODUCTION CHECK

- Declarative only (`apply`) — imperative for quick tests
- `kubectl diff` before every apply

IN DOCKER / COMPOSE

- `kubectl run` ↔ `docker run` · `apply -f` ↔ `compose up`
- `--dry-run -o yaml` ↔ `compose config`

THREE APPROACHES

STYLE	USE FOR
`create deploy web …`	Experiments
`create -f x.yaml`	Rare; fails if exists
`apply -f dir/`	Everything real
`replace --force -f`	Immutable changes

PATCH --TYPE

TYPE	LISTS ARE…
`strategic`	Merged by key (default)
`merge`	Replaced whole
`json`	Edited by RFC 6902 ops
`--subresource=scale`	Patch only scale/status

DRY RUN

FLAG	MEANING
`=client`	Render locally
`=server`	Validate + admission
`--validate=strict`	Reject unknown fields
`diff -f`	Compare file to live

⚠ COMMON MISTAKES

- Hotfixing with `edit` — the next `apply` reverts it
- `create -f` in CI — reruns fail with `AlreadyExists`
- Expecting `apply` to delete objects removed from Git
- Hand-writing YAML from memory — generate it to avoid typos

✓ TIPS & BEST PRACTICES

- Run `kubectl diff` before applying to shared clusters
- `--dry-run=server` catches policy rejections early
- Copy any live hotfix back into Git right away
- Name the `--field-manager` per tool in CI

QUICK SUMMARY

- `create / run / expose` — imperative
- `apply -f` — declarative, idempotent
- `--dry-run=client -o yaml` — make YAML
- `diff` — preview; `edit / patch` — live drift

Sheet 06

Manifests, Labels & Selectors

apiVersion / kind / metadata / spec · --- · labels · selectors · annotations · app.kubernetes.io/*

ANATOMY OF A MANIFEST

Every object has `apiVersion`, `kind`, `metadata` and `spec` (what you want); Kubernetes adds `status` (what it has). **Labels** are key/value tags that **selectors** match: that's how a Service finds its Pods and a Deployment knows which Pods it owns. **Annotations** hold non-identifying data for tools.

NOTES

- Core kinds use `v1`; others `group/version`
- Label values ≤ 63 chars: letters, digits, `- _ .`
- Annotations can hold any string, up to 256 KB total
- Keys may carry a DNS prefix: `app.kubernetes.io/name`

A LABELED MANIFEST

```yaml
apiVersion: apps/v1                # group/version
kind: Deployment
metadata:
  name: web                        # unique per kind + namespace
  namespace: shop
  labels:                          # identifying, selectable
    app.kubernetes.io/name: web
    tier: frontend
  annotations:                     # free-form, for tools
    team: storefront@acme.io
spec:
  selector:                        # which pods it owns
    matchLabels: { app.kubernetes.io/name: web }
    matchExpressions:
      - { key: tier, operator: In, values: [frontend, edge] }
  template: { ... }                # pod template (Sheet 08)
---                                # next object, same file
```

CODE EXPLAINED

- → `apiVersion` — look it up in `api-resources`
- → `matchLabels` — every pair must match
- → `operator` — In, NotIn, Exists, DoesNotExist
- → `---` — separates documents in one file
- → `labels` on metadata — lets `get -l` find the Deployment
- → `{ ... }` — YAML flow style, same as indented maps

PRODUCTION CHECK

- Standard `app.kubernetes.io/*` labels on everything
- Selectors are immutable on Deployments — pick them once

IN DOCKER / COMPOSE

- Labels ↔ Docker `--label` / Compose `labels:`
- Selectors have no Docker equivalent

-L SELECTORS

EXPRESSION	MATCHES
`app=web`	Equals
`app!=web`	Not equal / missing
`'env in (a,b)'`	Any listed value
`tier`	Key exists
`'!tier'`	Key missing
`'env notin (dev)'`	None of the values

LABEL COMMANDS

KUBECTL …	DOES
`label po web env=prod`	Add
`… env=dev --overwrite`	Change
`label po web env-`	Remove
`annotate deploy web k=v`	Annotate
`get po -L env`	Show as column
`label po -l app=web x=1`	Bulk by selector

RECOMMENDED

APP.KUBERNETES.IO/	EXAMPLE
`name`	mysql
`instance`	mysql-orders
`version`	8.4.1
`part-of`	storefront
`managed-by`	Helm
`component`	database

⚠ COMMON MISTAKES

- Selector ≠ template labels — the API rejects the Deployment
- Editing a Deployment selector — it's immutable; recreate instead
- Two Deployments with overlapping selectors fight over Pods
- Putting URLs or long text in labels — values are limited to 63 chars

✓ TIPS & BEST PRACTICES

- Use the `app.kubernetes.io/*` labels; tools understand them
- Keep selectors small and stable
- Data goes in annotations, not labels
- `get pods -L app.kubernetes.io/version` shows versions at a glance

QUICK SUMMARY

- `apiVersion · kind · metadata · spec`
- `labels` — select; `annotations` — describe
- `-l` — filter; `label k-` — remove
- `matchLabels / matchExpressions` — how controllers find pods

Pods

THE SMALLEST UNIT

A **Pod** is one or more containers on one node sharing an IP (talk over `localhost`) and volumes. **Init containers** run to completion, in order, before the app. A **sidecar** is an init container with `restartPolicy: Always`: it starts first and runs beside the app. Pods are disposable; controllers create them from a template.

NOTES

- Recreated Pods get a new IP; use a Service
- `command` = ENTRYPOINT, `args` = CMD
- Native sidecars are stable since 1.33
- Most Pod spec fields are immutable; recreate instead of editing

SECURITY CHECK

- `:latest` + `Always` pulls whatever was pushed last
- Pin by digest

POD SPEC

```yaml
apiVersion: v1
kind: Pod
metadata: { name: api, labels: { app: api } }
spec:
  restartPolicy: Always            # Always | OnFailure | Never
  initContainers:
    - name: wait-db                # must exit 0 first
      image: busybox:1.36
      command: ["sh", "-c", "until nc -z db 5432; do sleep 2; done"]
    - name: logs                   # sidecar: runs alongside
      image: fluent/fluent-bit:3.1
      restartPolicy: Always
  containers:
    - name: api
      image: ghcr.io/acme/api:1.4.2
      command: ["/app/server"]     # overrides ENTRYPOINT
      args: ["--port=8080"]        # overrides CMD
      ports: [{ containerPort: 8080, name: http }]
```

CODE EXPLAINED

- → `restartPolicy` — Deployments require `Always`
- → `initContainers` — setup, waits, migrations
- → `restartPolicy: Always` on init — native sidecar
- → `name: http` — reference by name in Services/probes
- → `until nc -z` — blocks the app until the DB port answers
- → `image` — pin a tag or `@sha256:` digest

PRODUCTION CHECK

- Never run bare Pods — use a controller
- Set `imagePullPolicy` and a pinned tag

IN DOCKER / COMPOSE

- Pod ↔ one Compose service (+ sidecars)
- `command` / `args` ↔ `ENTRYPOINT` / `CMD`

POD PHASES

PHASE	MEANS
`Pending`	Not scheduled or still pulling
`Running`	On a node, a container is up
`Succeeded`	All exited 0, no restart
`Failed`	All stopped, one failed
`Unknown`	Node stopped reporting

STATUS REASONS

REASON	CAUSE
`ImagePullBackOff`	Bad tag or no auth
`CrashLoopBackOff`	Keeps exiting
`OOMKilled`	Over memory limit
`Completed`	Exited 0
`ContainerCreating`	Pulling, mounting volumes

IMAGEPULLPOLICY

VALUE	PULLS
`IfNotPresent`	If missing (pinned tags)
`Always`	Every start (`:latest`)
`Never`	Local only

⚠ COMMON MISTAKES

- Bare Pod for an app — not rescheduled when its node dies
- `:latest` tags — nodes may run different builds
- Setting `command` skips the image's entrypoint script
- Two containers binding one port — they share a network namespace

✓ TIPS & BEST PRACTICES

- One main process per container; helpers as sidecars
- `kubectl run api --image=… --dry-run=client -o yaml`
- Name ports and reference them by name
- `env` from ConfigMaps/Secrets: Sheet 15

QUICK SUMMARY

- `Pod` — containers sharing IP + volumes
- `initContainers` — run first, in order
- `command / args` — ENTRYPOINT / CMD
- `restartPolicy: Always` init container — native sidecar

Deployments & Rollouts

STATELESS APPS, UPDATED SAFELY

A **Deployment** owns **ReplicaSets**, which keep N identical Pods running. Changing the pod `template` creates a new ReplicaSet and **rolls** pods over gradually; the old one is kept at 0 replicas so you can **undo**. Changing only `replicas` scales without a rollout.

NOTES

- Pod names: `web-<template-hash>-<id>`
- Rollouts advance only as new pods turn **Ready**
- Stuck after `progressDeadlineSeconds` (600) = failed, not rolled back
- Need stable names or per-pod disks? Use a StatefulSet (Sheet 09)

DEPLOYMENT

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  revisionHistoryLimit: 5        # old ReplicaSets kept for undo
  selector:
    matchLabels: { app: web }    # = template labels; immutable
  strategy:
    type: RollingUpdate          # or Recreate
    rollingUpdate:
      maxSurge: 25%              # extra pods during update
      maxUnavailable: 0          # never below 3 ready
  template:
    metadata: { labels: { app: web } }
    spec:
      containers: [{ name: web, image: nginx:1.27 }]
```

CODE EXPLAINED

- → `replicas` — leave out if an HPA scales it
- → `revisionHistoryLimit` — default 10
- → `maxUnavailable: 0` — zero downtime, needs spare room
- → `template` — any change triggers a rollout
- → `selector` — must match template labels exactly
- → `maxSurge: 25%` — of 3 replicas rounds up to 1 extra pod

PRODUCTION CHECK

- `maxUnavailable: 0` for zero-downtime rollouts
- Readiness probes gate each step

IN DOCKER / COMPOSE

- Deployment ↔ Swarm service / Compose `deploy.replicas`
- `rollout undo` ↔ `service rollback`

ROLLOUT COMMANDS (KUBECTL ROLLOUT …)

COMMAND	DOES
`status deploy/web`	Wait; non-zero exit on failure
`history deploy/web`	List revisions
`undo deploy/web`	Previous revision
`undo deploy/web --to-revision=2`	Specific revision
`restart deploy/web`	New pods, same spec
`pause / resume deploy/web`	Batch edits into one rollout

STRATEGIES

TYPE	BEHAVIOR
`RollingUpdate`	Gradual; versions overlap
`Recreate`	All down, then up

SCALE & UPDATE

KUBECTL …	DOES
`scale deploy/web --replicas=5`	Scale
`set image deploy/web web=nginx:1.28`	Roll out

⚠ COMMON MISTAKES

- Editing a ConfigMap and expecting a restart — `rollout restart`
- No readiness probe — rollout "succeeds" before pods work
- `maxUnavailable: 0` on a full cluster — surge pods stay Pending
- Treating `rollout undo` as a fix — Git still has the bad version

✓ TIPS & BEST PRACTICES

- Gate CI on `rollout status --timeout=5m`
- `Recreate` when two versions can't coexist
- After `undo`, fix the manifest in Git too
- Pair with a PodDisruptionBudget for node drains (Sheet 19)

QUICK SUMMARY

- `Deployment → ReplicaSet → Pods`
- `maxSurge / maxUnavailable` — speed vs capacity
- `rollout status / undo / restart`
- `template` change — rollout; `replicas` — scale only

StatefulSets & DaemonSets

IDENTITY AND PER-NODE WORKLOADS

A **StatefulSet** gives each pod a **stable name** (`db-0`, `db-1`), a stable DNS entry via a **headless Service**, and its **own PVC** that follows it across reschedules. Pods start in order and stop in reverse. A **DaemonSet** runs exactly **one pod per node** (or per matching node): log shippers, metrics agents, CNI and storage drivers.

NOTES

- StatefulSets don't make an app clustered; the app must handle its own replication
- PVCs from `volumeClaimTemplates` survive scale-down and deletion by default
- DaemonSets ignore `replicas`; new nodes get a pod automatically

SECURITY CHECK

- DaemonSets often run privileged — review them
- Only trusted agents on every node

STATEFULSET

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: db }
spec:
  serviceName: db                  # headless Service (clusterIP: None)
  replicas: 3                      # db-0, db-1, db-2, created in order
  podManagementPolicy: OrderedReady    # or Parallel
  updateStrategy: { rollingUpdate: { partition: 0 } }
  selector: { matchLabels: { app: db } }
  template: { ... }                # container mounts volume "data"
  volumeClaimTemplates:            # one PVC per pod: data-db-0 …
    - metadata: { name: data }
      spec:
        accessModes: [ReadWriteOnce]
        resources: { requests: { storage: 10Gi } }
```

DAEMONSET

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: node-exporter, namespace: monitoring }
spec:
  selector: { matchLabels: { app: node-exporter } }
  template: { ... }                # + nodeSelector / tolerations
```

CODE EXPLAINED

- → `serviceName` — gives pods DNS like `db-0.db.shop.svc.cluster.local`
- → `OrderedReady` — db-1 waits for db-0 to be Ready; `Parallel` doesn't
- → `partition: 2` — only pods with ordinal ≥ 2 update (canary)
- → `volumeClaimTemplates` — PVC named `<tmpl>-<pod>`
- → `maxUnavailable` — DaemonSet pods replaced per step
- → `nodeSelector` — limit a DaemonSet to some nodes

PRODUCTION CHECK

- StatefulSets need a backup plan, not just PVCs
- Prefer managed databases over StatefulSets

IN DOCKER / COMPOSE

- StatefulSet ↔ a Compose service with a named volume
- DaemonSet ↔ Swarm `global` mode

WHICH CONTROLLER?

FEATURE	DEPLOYMENT	STATEFULSET	DAEMONSET
Pod names	Random	`name-0…N`	Random, 1/node
Storage	Shared or none	PVC per pod	Usually hostPath
Scaling	`replicas`	`replicas`, ordered	Follows nodes
Use for	Web/API	DBs, Kafka, ZooKeeper	Agents, drivers

COMMANDS

KUBECTL …	DOES
`scale sts db --replicas=5`	Adds db-3, db-4
`rollout status sts/db`	Wait for update
`rollout restart ds/node-exporter`	Bounce all
`delete sts db --cascade=orphan`	Keep pods

⚠ COMMON MISTAKES

- Forgetting the headless Service — stable pod DNS names don't resolve
- Expecting PVCs to vanish on scale-down — they stay (and keep billing)
- Scaling a database StatefulSet and assuming data replicates by itself

✓ TIPS & BEST PRACTICES

- Use an operator (CloudNativePG, Strimzi) for serious databases
- Set `persistentVolumeClaimRetentionPolicy` to control PVC cleanup
- `Parallel` speeds up apps that don't need ordering

QUICK SUMMARY

- `StatefulSet` — stable names, ordered, PVC per pod
- `serviceName` — headless Service for pod DNS
- `partition` — staged rollouts
- `DaemonSet` — one pod per node

Jobs & CronJobs

RUN TO COMPLETION

A **Job** runs pods until a set number **succeed**, retrying failures up to `backoffLimit`. Use it for migrations, batch processing and one-off tasks. A **CronJob** creates a Job on a **cron schedule**, with rules for overlapping runs and how much history to keep.

NOTES

- Job pods need `restartPolicy: Never` or `OnFailure`
- Missed CronJob runs beyond `startingDeadlineSeconds` are skipped
- `timeZone` takes IANA names; otherwise the controller's zone is used

JOB

```yaml
apiVersion: batch/v1
kind: Job
metadata: { name: migrate }
spec:
  completions: 1                # successes needed
  backoffLimit: 4               # retries before Failed
  activeDeadlineSeconds: 600    # hard time limit for the Job
  ttlSecondsAfterFinished: 3600 # auto-delete 1h after finishing
  template:
    spec:
      restartPolicy: Never
      containers: [{ name: m, image: acme/api:1.4.2, command: ["./migrate"] }]
```

CRONJOB

```yaml
apiVersion: batch/v1
kind: CronJob
metadata: { name: nightly-report }
spec:
  schedule: "30 2 * * *"        # 02:30 every day
  timeZone: America/Chicago
  concurrencyPolicy: Forbid     # skip if last run still going
  jobTemplate: { spec: { template: { ... } } }
```

CODE EXPLAINED

- → `completions: 5, parallelism: 2` — 5 successes, 2 at a time
- → `backoffLimit` — default 6
- → `activeDeadlineSeconds` — beats `backoffLimit`; kills running pods
- → `ttlSecondsAfterFinished` — keeps the namespace tidy
- → `Never` — new pod per retry; `OnFailure` restarts in place
- → `jobTemplate` — a full Job spec

PRODUCTION CHECK

- `ttlSecondsAfterFinished` to clean up
- `concurrencyPolicy: Forbid` for jobs that can't overlap

IN DOCKER / COMPOSE

- Job ↔ `docker run --rm` · CronJob ↔ cron + `docker run`
- `backoffLimit` ↔ `--restart on-failure:N`

CRON FIELDS

FIELD	RANGE
`minute`	0–59
`hour`	0–23
`day of month`	1–31
`month`	1–12
`day of week`	0–6 (Sun = 0)

CONCURRENCYPOLICY

VALUE	IF STILL RUNNING
`Allow`	Start another (default)
`Forbid`	Skip this run
`Replace`	Kill old, start new
`suspend: true`	Pause the schedule

COMMANDS

KUBECTL …	DOES
`create job now --from=cronjob/x`	Run now
`logs job/migrate`	Logs of a job pod
`wait --for=condition=complete job/m`	Block in CI
`get jobs -w`	Watch progress

⚠ COMMON MISTAKES

- `restartPolicy: Always` in a Job — rejected by the API
- Non-idempotent job code — retries run it twice (double charges, dup emails)

✓ TIPS & BEST PRACTICES

- Test a CronJob instantly with `create job --from=cronjob/…`
- Use `Forbid` for jobs that must not overlap
- `completionMode: Indexed` gives each pod a `JOB_COMPLETION_INDEX`

QUICK SUMMARY

- `Job` — run until N successes
- `backoffLimit / activeDeadlineSeconds` — limits
- `CronJob` — Job on a schedule
- `concurrencyPolicy` — overlap rules

Health Probes & Pod Lifecycle

livenessProbe · readinessProbe · startupProbe · httpGet / tcpSocket / exec / grpc · preStop · terminationGracePeriodSeconds

TELLING KUBERNETES WHAT HEALTHY MEANS

The kubelet runs **probes** against each container. A failing **readiness** probe removes the pod from Service endpoints (no traffic) without restarting it. A failing **liveness** probe restarts the container. A **startup** probe holds off the other two until a slow app has booted. On shutdown, pods get `SIGTERM`, a grace period, then `SIGKILL`.

NOTES

- No probes: the pod is Ready as soon as the process starts
- Liveness should check the process only, never databases or other services
- Readiness can (carefully) check dependencies the pod truly needs
- Probe traffic comes from the kubelet on the node, not from the Service

PROBES & SHUTDOWN

```
spec:
  terminationGracePeriodSeconds: 30  # SIGTERM → wait → SIGKILL
  containers:
    - name: api
      image: ghcr.io/acme/api:1.4.2
      ports: [{ name: http, containerPort: 8080 }]
      startupProbe:                  # protects slow starters
        httpGet: { path: /healthz, port: http }
        periodSeconds: 5
        failureThreshold: 30         # up to 150s to boot
      livenessProbe:                 # fail → restart container
        httpGet: { path: /healthz, port: http }
        periodSeconds: 10
        timeoutSeconds: 2
      readinessProbe:                # fail → no Service traffic
        httpGet: { path: /ready, port: http }
        periodSeconds: 5
      lifecycle:
        preStop: { exec: { command: ["sleep", "5"] } }   # let LBs drain
```

CODE EXPLAINED

- → `startupProbe` — liveness/readiness start only after it passes
- → `failureThreshold × periodSeconds` — time before giving up
- → `port: http` — named container port
- → `timeoutSeconds` — default is just 1s
- → `preStop` — runs before SIGTERM; counts against the grace period
- → `sleep 5` — endpoints update before the app stops accepting

PRODUCTION CHECK

- Readiness on every serving pod; liveness only if it can deadlock
- `preStop` sleep so the LB drains first

IN DOCKER / COMPOSE

- Probes ↔ Docker `HEALTHCHECK`
- `terminationGracePeriodSeconds` ↔ `docker stop -t`

PROBE HANDLERS

HANDLER	PASSES WHEN
`httpGet`	Status 200–399
`tcpSocket`	Port accepts a connection
`exec`	Command exits 0
`grpc`	gRPC health check SERVING

FIELDS & DEFAULTS

FIELD	DEFAULT
`initialDelaySeconds`	0
`periodSeconds`	10
`timeoutSeconds`	1
`failureThreshold`	3
`successThreshold`	1

SHUTDOWN ORDER

STEP	WHAT HAPPENS
1	Pod marked Terminating, leaves endpoints
2	`preStop` hook runs
3	`SIGTERM` to PID 1
4	`SIGKILL` after grace period

⚠ COMMON MISTAKES

- Liveness probe that checks the database — a DB blip restarts every pod
- Same endpoint and timing for liveness and readiness — restarts when you only wanted to shed load
- Liveness with no startup probe on a slow app — restart loop before it ever boots

✓ TIPS & BEST PRACTICES

- Liveness: cheap "is the process alive"; readiness: "can I serve now"
- Raise `timeoutSeconds` to 2–5s for real-world endpoints
- Use exec-form ENTRYPOINT so your app is PID 1 and gets SIGTERM

QUICK SUMMARY

- `readiness` — gates traffic
- `liveness` — restarts the container
- `startup` — delays the others for slow boots
- `preStop + grace period` — clean shutdown

Services & DNS

A STABLE ADDRESS FOR CHANGING PODS

Pods come and go with new IPs; a **Service** gives a set of pods (chosen by **label selector**) one stable virtual IP and DNS name and load-balances across the **Ready** ones. kube-proxy (or the CNI) programs the routing on every node. **CoreDNS** answers `<svc>.<ns>.svc.cluster.local`.

NOTES

- Selector matches nothing → Service exists but has no endpoints
- Pods in the same namespace can just use the short name: `http://web`
- EndpointSlices list the backing pod IPs; the old Endpoints API is deprecated

SECURITY CHECK

- NodePort opens the port on every node
- Prefer ClusterIP + Ingress

SERVICE

```yaml
apiVersion: v1
kind: Service
metadata: { name: web, namespace: shop }
spec:
  type: ClusterIP                # default; see types table
  selector: { app: web }         # pods that get traffic
  ports:
    - name: http
      port: 80                   # what clients connect to
      targetPort: http           # container port (number or name)
```

CODE EXPLAINED

- → `selector` — must match pod labels, not Deployment labels
- → `port` vs `targetPort` — Service side vs container side
- → `nodePort` — set only for NodePort/LoadBalancer (30000–32767)
- → `endpointslices` — empty means selector or readiness problem

PRODUCTION CHECK

- ClusterIP inside, Ingress / Gateway outside
- One LoadBalancer per cluster, not per service

CHECK IT

```bash
kubectl get svc,endpointslices -n shop -l app=web
kubectl port-forward svc/web 8080:80 -n shop      # http://localhost:8080
```

IN DOCKER / COMPOSE

- Service ↔ `-p` publishing + Compose service DNS
- CoreDNS names ↔ Compose service names

SERVICE TYPES

TYPE	REACHABLE FROM
`ClusterIP`	Inside the cluster only
`NodePort`	Every node's IP on a high port
`LoadBalancer`	Cloud LB → NodePort → pods
`ExternalName`	DNS CNAME to an outside host
`clusterIP: None`	Headless: DNS returns pod IPs

DNS NAMES

NAME	RESOLVES TO
`web`	Same-namespace Service
`web.shop`	Service in namespace shop
`web.shop.svc.cluster.local`	Fully qualified
`db-0.db.shop.svc…`	One StatefulSet pod

USEFUL FIELDS

FIELD	EFFECT
`externalTrafficPolicy: Local`	Keep client IP
`sessionAffinity: ClientIP`	Sticky by IP
`internalTrafficPolicy: Local`	Node-local only
`loadBalancerSourceRanges`	IP allowlist

⚠ COMMON MISTAKES

- Selector copied from the Deployment's metadata labels instead of pod template labels
- `targetPort` not matching the container's listening port — connection refused
- One LoadBalancer Service per app — costly; share an Ingress/Gateway

✓ TIPS & BEST PRACTICES

- Name ports and use `targetPort: http` so port changes live in one place
- `kubectl get endpointslices` first when a Service "doesn't work"
- `port-forward` for local testing; no Service type change needed

QUICK SUMMARY

- `Service` — stable IP + DNS over Ready pods
- `ClusterIP / NodePort / LoadBalancer` — widening reach
- `port → targetPort` — client side → container side
- `svc.ns.svc.cluster.local` — CoreDNS name

Ingress & Gateway API

HTTP ROUTING INTO THE CLUSTER

Ingress routes HTTP(S) by host and path to Services, through an **ingress controller** you install. It's stable but frozen. **Gateway API** is its successor: roles are split into `GatewayClass` (provider), `Gateway` (listeners, ports, TLS; owned by ops) and `HTTPRoute` (owned by app teams), with header matching, traffic weights and more built in.

NOTES

- Both need a controller: Envoy Gateway, Istio, Cilium, Traefik, cloud LBs…
- Community `ingress-nginx` is retired; plan a migration
- Gateway API CRDs may need installing

SECURITY CHECK

- Ingress controllers are internet-facing — keep them patched
- Restrict annotations that inject config

INGRESS

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: shop }
spec:
  ingressClassName: traefik          # which controller handles it
  tls: [{ hosts: [shop.acme.io], secretName: shop-tls }]
  rules:
    - host: shop.acme.io
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend: { service: { name: api, port: { number: 80 } } }
```

CODE EXPLAINED

- → `ingressClassName` — picks the controller; list with `get ingressclass`
- → `tls.secretName` — a `kubernetes.io/tls` Secret
- → `pathType: Prefix` — `/api` matches `/api/x`, not `/apix`
- → `parentRefs` — the Gateway must allow routes from this namespace
- → `weight` — split traffic without a service mesh

GATEWAY API: HTTPROUTE

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: shop }
spec:
  parentRefs: [{ name: public-gw }]  # attach to a Gateway
  rules:
    - matches: [{ path: { type: PathPrefix, value: /api } }]
      backendRefs:
        - { name: api, port: 80, weight: 90 }
        - { name: api-v2, port: 80, weight: 10 }   # 10% canary
```

PRODUCTION CHECK

- TLS on every public host (cert-manager)
- Gateway API for new clusters

IN DOCKER / COMPOSE

- Ingress ↔ a reverse proxy container (Traefik / nginx)
- Compose: Traefik labels per service

GATEWAY API OBJECTS

KIND	OWNED BY · HOLDS
`GatewayClass`	Provider · which controller
`Gateway`	Platform team · listeners, ports, TLS
`HTTPRoute`	App team · hosts, paths, backends
`ReferenceGrant`	Allows cross-namespace refs

INGRESS PATHTYPE

VALUE	MATCHES
`Exact`	Only that path
`Prefix`	Path segments
`ImplementationSpecific`	Controller-defined

⚠ COMMON MISTAKES

- No controller installed — Ingress objects are accepted and do nothing
- Missing `ingressClassName` with no default class — ignored
- Controller-specific annotations don't port between controllers

✓ TIPS & BEST PRACTICES

- New setups: start with Gateway API
- `kubectl describe httproute shop` → status shows Accepted / ResolvedRefs
- Automate TLS with cert-manager

QUICK SUMMARY

- `Ingress` — host/path → Service, frozen API
- `Gateway` — listeners, owned by ops
- `HTTPRoute` — routing rules, owned by apps
- controller required for both

Network Policies

A FIREWALL FOR PODS

By default every pod can reach every other pod in the cluster. A **NetworkPolicy** selects pods and lists the traffic **allowed** in (`ingress`) or out (`egress`). Once any policy selects a pod for a direction, everything not explicitly allowed in that direction is denied. Policies only add allows, so their order doesn't matter.

NOTES

- Enforced by the **CNI plugin**; without support, policies are silently ignored
- Selects by labels, namespaces and CIDRs, at L3/L4 (no HTTP paths)
- Replies to allowed connections are allowed automatically
- Deny egress → remember to allow DNS (port 53 to kube-system)

DEFAULT DENY + ALLOW

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec:
  podSelector: {}                  # every pod in the namespace
  policyTypes: [Ingress, Egress]   # nothing in or out by default
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api-from-web, namespace: shop }
spec:
  podSelector: { matchLabels: { app: api } }    # applies to api pods
  ingress:
    - from:
        - podSelector: { matchLabels: { app: web } }       # this ns
        - namespaceSelector: { matchLabels: { team: ops } } # OR these
      ports: [{ protocol: TCP, port: 8080 }]
```

CODE EXPLAINED

- → `podSelector: {}` — empty selector = all pods
- → `policyTypes` — directions isolated; defaults from the rules present
- → `from:` list items — OR'ed together
- → `podSelector` alone — same namespace as the policy
- → `ports` — applies to every `from` above it

PRODUCTION CHECK

- Default-deny per namespace, then allow
- Test policies in staging — the CNI must support them

IN DOCKER / COMPOSE

- NetworkPolicy ↔ separate user-defined networks
- Compose: `internal: true` networks

AND VS OR

SHAPE	MEANING
`-podSel / -nsSel`	Two items: OR
`-nsSel + podSel`	One item: AND
`-ipBlock: {cidr}`	IP ranges (+ except)

COMMON POLICIES

POLICY	RULE
`Deny all ingress`	Select all, no rules
`Allow DNS`	Egress 53 to kube-system
`Allow ingress ctrl`	Select its namespace
`Same ns only`	`from: [podSelector: {}]`

CNI SUPPORT

CNI	POLICIES
`Calico`	✔
`Cilium`	✔ (+L7 own CRD)
`Antrea`	✔
`Flannel alone`	✖

⚠ COMMON MISTAKES

- Applying policies on a CNI that doesn't enforce them — false sense of security
- Default-deny egress without a DNS rule — every lookup times out
- One extra `-` turns AND into OR and opens access
- Using `namespaceSelector` on a label the namespace lacks

✓ TIPS & BEST PRACTICES

- Select namespaces by the built-in `kubernetes.io/metadata.name` label
- Start with default-deny per namespace, then add allows
- Test with `kubectl exec … -- nc -zv api 8080`
- Hubble (Cilium) or Calico flow logs show dropped traffic

QUICK SUMMARY

- `NetworkPolicy` — allow-list per pod selector
- `podSelector: {}` + no rules — default deny
- list items OR, keys in one item AND
- CNI must support it

ConfigMaps & Secrets

CONFIGURATION OUTSIDE THE IMAGE

A **ConfigMap** holds non-secret key/value config; a **Secret** holds sensitive values and is handled a bit more carefully (RBAC, tmpfs mounts, optional encryption at rest). Pods consume both as **environment variables** or as **files** in a mounted volume. Same image, different config per environment.

NOTES

- Secret values are **base64-encoded, not encrypted**
- Env vars are read once at start; mounted files update (≈1 min), except `subPath`
- Size limit: 1 MiB per ConfigMap or Secret
- `immutable: true` blocks edits and lightens API server load

SECURITY CHECK

- Secrets are base64, not encrypted
- Enable encryption at rest; limit `get secrets` in RBAC

CREATE

```
kubectl create cm app-cfg --from-literal=LOG_LEVEL=info --from-file=app.yaml
kubectl create cm env-cfg --from-env-file=.env
kubectl create secret generic db --from-literal=pw='s3cr3t!'
kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key
kubectl get secret db -o jsonpath='{.data.pw}' | base64 -d      # s3cr3t!
```

CONSUME

```
spec:
  containers:
    - name: api
      envFrom: [{ configMapRef: { name: env-cfg } }]   # every key → env
      env:
        - name: DB_PASSWORD
          valueFrom: { secretKeyRef: { name: db, key: pw } }
      volumeMounts: [{ name: cfg, mountPath: /etc/app, readOnly: true }]
  volumes:
    - name: cfg
      configMap: { name: app-cfg }      # each key becomes a file
```

CODE EXPLAINED

- → `--from-file` — key = filename, value = contents
- → `--from-env-file` — one key per `KEY=value` line
- → `secret tls` — keys `tls.crt` and `tls.key`
- → `envFrom` — bulk import; `prefix:` avoids clashes
- → `secretKeyRef` — one key into one env var
- → `/etc/app/app.yaml` — file from the ConfigMap key

PRODUCTION CHECK

- Secrets from an external store (External Secrets, CSI)
- `immutable: true` for config that shouldn't change

IN DOCKER / COMPOSE

- ConfigMap ↔ Compose `configs:` · Secret ↔ `secrets:`
- `envFrom` ↔ `env_file`

CONSUMING

METHOD	LIVE UPDATE?
`env valueFrom`	✖ restart
`envFrom`	✖ restart
`volume mount`	✔ ~1 min
`volume + subPath`	✖

SECRET TYPES

TYPE:	FOR
`Opaque`	Anything (default)
`kubernetes.io/tls`	Cert + key
`kubernetes.io/dockerconfigjson`	Registry auth
`kubernetes.io/basic-auth`	User + password

KEEPING SECRETS SAFE

TOOL	DOES
`EncryptionConfiguration`	Encrypt in etcd
`RBAC on secrets`	Limit get/list
`External Secrets`	Sync from Vault/AWS/GCP
`Sealed Secrets / SOPS`	Encrypted in Git

⚠ COMMON MISTAKES

- Committing Secret YAML to Git — base64 is readable by anyone
- Expecting env vars to change after editing a ConfigMap — restart the pods
- `subPath` mounts never update
- Giving `list secrets` broadly — it returns every value in the namespace

✓ TIPS & BEST PRACTICES

- Add a config hash annotation to the pod template so changes roll out
- Use `stringData:` in YAML to write plain text; the API encodes it
- Registry auth: `create secret docker-registry` + `imagePullSecrets`
- Prefer an external secret manager for production

QUICK SUMMARY

- `ConfigMap` — plain config
- `Secret` — sensitive, base64 only
- `env / envFrom` — read at start
- `volume` — files, updated live

Sheet 16

Volumes & Persistent Storage

emptyDir · hostPath · PersistentVolume · PersistentVolumeClaim · StorageClass · RWO / RWX / RWOP · CSI

DATA THAT OUTLIVES CONTAINERS

A **volume** is a directory mounted into containers. `emptyDir` lives as long as the pod. For data that must survive the pod, a **PersistentVolumeClaim** (PVC) requests storage; a **StorageClass** tells a **CSI driver** to provision a matching **PersistentVolume** (PV) on demand, such as an EBS, Persistent Disk or Azure Disk volume.

NOTES

- PVCs are namespaced; PVs and StorageClasses are cluster-scoped
- One class can be the default (annotation `is-default-class`)
- Block disks are usually `ReadWriteOnce`: one node at a time
- Shared RWX needs a file system: EFS, Filestore, Azure Files, NFS

SECURITY CHECK

- `hostPath` gives pods host files
- Block it with Pod Security `restricted`

PVC + POD

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: pg-data }
spec:
  accessModes: [ReadWriteOnce]        # one node at a time
  storageClassName: fast-ssd          # omit → default class
  resources: { requests: { storage: 20Gi } }
---
# inside a Pod / pod template spec:
volumes:
  - { name: data, persistentVolumeClaim: { claimName: pg-data } }
  - { name: scratch, emptyDir: { sizeLimit: 1Gi } }   # dies with pod
containers:
  - name: db
    image: postgres:17
    volumeMounts:
      - { name: data, mountPath: /var/lib/postgresql/data }
      - { name: scratch, mountPath: /tmp }
```

CODE EXPLAINED

- → `accessModes` — must be supported by the driver
- → `storageClassName: ""` — bind only to a pre-made PV
- → `requests.storage` — grow later if the class allows expansion
- → `claimName` — PVC must be in the pod's namespace
- → `emptyDir.medium: Memory` — tmpfs, counts toward memory

PRODUCTION CHECK

- A `StorageClass` with backups and the right reclaim policy
- `Retain` for data you can't lose

IN DOCKER / COMPOSE

- PVC ↔ named volume · `hostPath` ↔ bind mount
- `emptyDir` ↔ anonymous volume / tmpfs

VOLUME TYPES

TYPE	LIFETIME / USE
`emptyDir`	Pod lifetime; scratch, caches
`persistentVolumeClaim`	Independent; databases
`configMap / secret`	Config files (Sheet 15)
`hostPath`	Node's disk; agents only

ACCESS MODES

MODE	MEANING
`RWO`	Read-write, one node
`RWOP`	Read-write, one pod
`ROX`	Read-only, many nodes
`RWX`	Read-write, many nodes

STORAGECLASS FIELDS

FIELD	VALUES
`provisioner`	`ebs.csi.aws.com…`
`reclaimPolicy`	Delete | Retain
`volumeBindingMode`	WaitForFirstConsumer
`allowVolumeExpansion`	true

⚠ COMMON MISTAKES

- PVC stuck `Pending` — no default class, or the class can't satisfy the access mode
- `reclaimPolicy: Delete` on precious data — deleting the PVC deletes the disk
- Two pods on different nodes sharing an RWO volume — second one won't start

✓ TIPS & BEST PRACTICES

- Use `WaitForFirstConsumer` so disks are created in the pod's zone
- Resize: edit `spec.resources.requests.storage` on the PVC
- Back up volumes with VolumeSnapshots or Velero

QUICK SUMMARY

- `emptyDir` — scratch, pod lifetime
- `PVC` — request; `PV` — actual disk
- `StorageClass` — dynamic provisioning via CSI
- `RWO / RWX / RWOP` — who can mount

Requests, Limits & Quotas

SIZING CONTAINERS

Requests are what the scheduler reserves: a pod only lands on a node with that much unrequested CPU and memory. **Limits** are hard caps enforced at runtime: going over the CPU limit gets the container **throttled**; going over the memory limit gets it **OOMKilled**. **LimitRange** sets per-container defaults in a namespace; **ResourceQuota** caps the namespace's totals.

NOTES

- Requests drive scheduling and autoscaling; set them on every container
- Many teams set a memory limit but no CPU limit, to avoid throttling
- With a quota on CPU/memory, pods without requests are rejected

RESOURCES & QUOTA

```yaml
containers:
  - name: api
    image: ghcr.io/acme/api:1.4.2
    resources:
      requests: { cpu: 250m, memory: 256Mi }   # reserved for scheduling
      limits:   { memory: 512Mi }              # hard cap (no CPU limit)
---
apiVersion: v1
kind: ResourceQuota                 # totals for the whole namespace
metadata: { name: shop-quota, namespace: shop }
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.memory: 32Gi
    pods: "50"
    persistentvolumeclaims: "10"
```

CODE EXPLAINED

- → `250m` — 250 millicores = a quarter of one CPU
- → `256Mi` — mebibytes (2^{20}); `M` is 10^6
- → Limit only → request defaults to the limit
- → `hard` — new pods that would exceed it are rejected
- → `kubectl describe quota -n shop` — used vs hard

PRODUCTION CHECK

- Requests on every container; memory limits too
- ResourceQuota per namespace

IN DOCKER / COMPOSE

- `resources.limits` ↔ `--memory` / `--cpus`
- QoS classes have no Docker equivalent

QOS CLASSES

CLASS	WHEN	EVICTED
`Guaranteed`	requests = limits, all containers	Last
`Burstable`	Some requests/limits set	Middle
`BestEffort`	Nothing set	First

GOING OVER

RESOURCE	RESULT
`CPU limit`	Throttled (slow, not killed)
`Memory limit`	OOMKilled, exit 137
`Node memory`	Eviction by QoS + usage
`Ephemeral disk`	Evicted

LIMITRANGE FIELDS

FIELD	SETS
`defaultRequest`	Request if missing
`default`	Limit if missing
`min / max`	Allowed range
`maxLimitRequestRatio`	Limit ÷ request cap

⚠ COMMON MISTAKES

- No requests — scheduler overpacks nodes and the HPA can't compute utilization
- Memory limit below real peak usage — random OOMKilled restarts
- Tight CPU limits on latency-sensitive apps — throttling spikes p99

✓ TIPS & BEST PRACTICES

- Measure with `kubectl top pods` before picking numbers
- Set memory request = limit for predictable memory
- Add LimitRange defaults so forgotten specs still get sane values

QUICK SUMMARY

- `requests` — reserved, used to schedule
- `limits` — CPU throttles, memory kills
- `LimitRange / ResourceQuota` — defaults / totals

Scheduling & Node Placement

CONTROLLING WHERE PODS LAND

The scheduler filters nodes that fit a pod's requests, then scores them. You steer it with **nodeSelector** and **node affinity** (pod wants certain nodes), **pod affinity / anti-affinity** (near or away from other pods), **topology spread** (even out across zones or nodes) and **taints**, which repel pods unless they carry a matching **toleration**.

NOTES

- `required…` rules are hard; `preferred…` rules are weighted hints
- `…IgnoredDuringExecution`: running pods aren't evicted if labels change
- A toleration allows a tainted node; it doesn't attract pods there

PLACEMENT RULES

```
spec:
  nodeSelector: { kubernetes.io/arch: arm64 }   # simple must-match
  affinity:
    podAntiAffinity:                    # keep replicas apart
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector: { matchLabels: { app: web } }
            topologyKey: kubernetes.io/hostname
  topologySpreadConstraints:            # even across zones
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector: { matchLabels: { app: web } }
  tolerations:                          # may use GPU nodes
    - { key: gpu, operator: Exists, effect: NoSchedule }
```

CODE EXPLAINED

- → `nodeSelector` — node must have every label
- → `topologyKey: hostname` — "apart" means different nodes
- → `maxSkew: 1` — zone pod counts differ by at most 1
- → `ScheduleAnyway` — soft; `DoNotSchedule` — hard
- → `operator: Exists` — any value of the key

PRODUCTION CHECK

- Spread replicas across zones
- Taints for special hardware (GPUs)

IN DOCKER / COMPOSE

- Affinity ↔ Swarm placement constraints
- `drain` ↔ Swarm `--availability drain`

NODE COMMANDS (KUBECTL …)

COMMAND	DOES
`cordon worker-1`	No new pods
`drain worker-1 --ignore-daemonsets --delete-emptydir-data`	Evict for maintenance
`uncordon worker-1`	Schedulable again
`taint nodes worker-1 gpu=true:NoSchedule`	Add taint
`taint nodes worker-1 gpu-`	Remove taint
`label nodes worker-1 disktype=ssd`	Add label

TAINT EFFECTS

EFFECT	PODS WITHOUT TOLERATION
`NoSchedule`	Not scheduled
`PreferNoSchedule`	Avoided if possible
`NoExecute`	Evicted too

WELL-KNOWN LABELS

LABEL	EXAMPLE
`kubernetes.io/hostname`	worker-1
`topology.kubernetes.io/zone`	us-east-1a
`node.kubernetes.io/instance-type`	m7g.large

⚠ COMMON MISTAKES

- Hard anti-affinity with more replicas than nodes — extra pods stay Pending
- Expecting a toleration to pin pods to tainted nodes — add affinity too
- `drain` without a PDB — every replica may be evicted at once

✓ TIPS & BEST PRACTICES

- Prefer topology spread over pod anti-affinity for spreading
- Dedicated nodes = taint + toleration + node affinity together
- `describe pod` Events show which filter failed

QUICK SUMMARY

- `nodeSelector / nodeAffinity` — pick nodes
- `taints + tolerations` — keep pods off
- `cordon / drain / uncordon` — maintenance

Autoscaling & Disruption Budgets

SCALING PODS AND NODES

The **HorizontalPodAutoscaler** (HPA) changes a workload's `replicas` to hold a target metric, usually CPU utilization *relative to requests*. The **VerticalPodAutoscaler** adjusts the requests themselves. When pods don't fit, **Cluster Autoscaler** or **Karpenter** adds nodes. A **PodDisruptionBudget** limits how many pods voluntary disruptions (drains, upgrades, scale-down) may take at once.

NOTES

- Resource metrics need **metrics-server** installed
- Remove `replicas` from Git manifests the HPA manages
- PDBs don't protect against node crashes (involuntary)

HPA + PDB

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: web }
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: web }
spec:
  minAvailable: 2                    # or maxUnavailable: 1
  selector: { matchLabels: { app: web } }
```

CODE EXPLAINED

- → `scaleTargetRef` — Deployment, StatefulSet, or any `/scale`
- → `averageUtilization: 70` — 70% of the CPU *request*
- → `behavior:` — tune scale-up/down speed (default 5 min down window)
- → `minAvailable` — drains wait while fewer would remain
- → `selector` — same pods as the Deployment
- → `maxReplicas` — a cost ceiling; set it deliberately

PRODUCTION CHECK

- PDBs for every multi-replica service
- HPA needs requests set to work

IN DOCKER / COMPOSE

- HPA ↔ manual `docker service scale`
- No autoscaling in plain Docker / Compose

AUTOSCALERS

TOOL	SCALES
HPA	Pod count from metrics
VPA	Pod requests (add-on)
KEDA	Pods from events/queues, to zero
Cluster Autoscaler	Node groups for Pending pods
Karpenter	Right-sized nodes on demand

COMMANDS (KUBECTL …)

COMMAND	DOES
`autoscale deploy web --max=10 --cpu-percent=70`	Create HPA
`get hpa -w`	Watch targets vs current
`describe hpa web`	Scaling events
`top pods`	Live CPU/memory
`get pdb`	Allowed disruptions
`get hpa web -o yaml`	Current metrics, conditions

⚠ COMMON MISTAKES

- HPA on pods with no CPU requests — target shows `<unknown>`
- HPA and VPA both acting on CPU — they fight; use one per resource
- PDB `minAvailable` = replicas — drains block forever
- Expecting instant scale-up — new nodes can take minutes to join

✓ TIPS & BEST PRACTICES

- Set `minReplicas` ≥ 2 for anything user-facing
- Scale on custom metrics (RPS, queue depth) via adapters or KEDA
- Run VPA in `Off` mode to get request recommendations
- Pair HPA with Cluster Autoscaler or Karpenter so new pods have room

QUICK SUMMARY

- `HPA` — replicas from metrics
- `Cluster Autoscaler` / `Karpenter` — nodes
- `PDB` — cap voluntary disruptions
- `VPA` — right-size requests

RBAC & ServiceAccounts

Role / ClusterRole · RoleBinding / ClusterRoleBinding · verbs · ServiceAccount · create token · auth can-i

WHO CAN DO WHAT

RBAC answers *can subject X do verb Y on resource Z*. A **Role** (namespaced) or **ClusterRole** lists allowed verbs on resources; a **RoleBinding** or **ClusterRoleBinding** grants it to users, groups or **ServiceAccounts**. ServiceAccounts are identities for pods and automation; humans usually come from your cloud IAM or OIDC.

NOTES

- RBAC is additive: no deny rules, only allows
- Every namespace has a `default` ServiceAccount with no extra rights
- Pod tokens are short-lived and auto-rotated (projected volume)

SERVICEACCOUNT, ROLE, BINDING

```yaml
apiVersion: v1
kind: ServiceAccount
metadata: { name: ci-deployer, namespace: shop }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role                          # ClusterRole = cluster-wide
metadata: { name: deployer, namespace: shop }
rules:
  - apiGroups: ["apps"]             # "" = core group (pods, services)
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "patch", "update"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: ci-deployer, namespace: shop }
subjects: [{ kind: ServiceAccount, name: ci-deployer, namespace: shop }]
roleRef: { kind: Role, name: deployer, apiGroup: rbac.authorization.k8s.io }
```

CODE EXPLAINED

- → `apiGroups` — from `kubectl api-resources` APIVERSION
- → `resources` — plural names; subresources like `pods/log`
- → `verbs` — see table; `*` means all
- → `roleRef` — immutable; delete and recreate to change
- → Pod uses it via `serviceAccountName: ci-deployer`

PRODUCTION CHECK

- One ServiceAccount per app, no default SA tokens
- `auth can-i` in reviews

IN DOCKER / COMPOSE

- RBAC ↔ nothing in Docker (socket access = full control)
- Swarm: no per-user roles either

SCOPE COMBINATIONS

ROLE + BINDING	GRANTS
`Role` + `RoleBinding`	One namespace
`ClusterRole` + `RoleBinding`	Reused rules, one ns
`ClusterRole` + `CRB`	Whole cluster

COMMANDS (KUBECTL …)

COMMAND	DOES
`auth can-i delete pods -n shop`	yes / no
`auth can-i --list -n shop`	Everything you can do
`… --as=system:serviceaccount:shop:ci-deployer`	Check as the SA
`create token ci-deployer --duration=1h`	Short-lived token
`create rolebinding v --clusterrole=view --user=amy -n shop`	Quick grant

BUILT-IN CLUSTERROLES

ROLE	ALLOWS
`cluster-admin`	Everything
`admin`	All in a ns, incl. RBAC
`edit`	Read/write, no RBAC
`view`	Read-only, no Secrets

⚠ COMMON MISTAKES

- Binding `cluster-admin` to CI — one leaked token owns the cluster
- Granting `list secrets` — returns every secret's contents
- Pods auto-mounting tokens they never use — set `automountServiceAccountToken: false`

✓ TIPS & BEST PRACTICES

- Test with `auth can-i … --as=` before handing out access
- Start from `view` / `edit` and add narrow Roles
- Use cloud workload identity (IRSA, GKE WI) instead of static keys

QUICK SUMMARY

- `Role / ClusterRole` — what is allowed
- `RoleBinding / ClusterRoleBinding` — to whom
- `auth can-i` — verify

Pod Security

securityContext · runAsNonRoot · readOnlyRootFilesystem · drop ALL · allowPrivilegeEscalation · seccomp · Pod Security Admission

LOCKING DOWN CONTAINERS

A container's **securityContext** controls the Linux user, capabilities, privilege escalation, seccomp profile and file-system writes. **Pod Security Admission** (built in) checks pods against the **Pod Security Standards**: `privileged`, `baseline` or `restricted`, chosen per namespace with labels. It replaced PodSecurityPolicy, removed in 1.25.

NOTES

- Pod-level `securityContext` sets defaults; container-level wins
- `runAsNonRoot` fails pods whose image runs as UID 0
- Need custom rules? Use Kyverno or OPA Gatekeeper

RESTRICTED NAMESPACE + HARDENED POD

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: restricted   # reject violators
    pod-security.kubernetes.io/warn: restricted      # warn on apply
---
spec:                                  # pod spec
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001                     # group owning mounted volumes
    seccompProfile: { type: RuntimeDefault }
  containers:
    - name: api
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: [ALL] }
```

CODE EXPLAINED

- → `enforce` — rejected at the API; `warn` — shown to kubectl
- → `runAsUser: 10001` — avoid UIDs that exist on the host
- → `RuntimeDefault` — the runtime's default syscall filter
- → `readOnlyRootFilesystem` — mount an `emptyDir` for `/tmp`
- → `drop: [ALL]` — add back only e.g. `NET_BIND_SERVICE`

PRODUCTION CHECK

- `restricted` Pod Security on app namespaces
- Read-only root filesystem by default

IN DOCKER / COMPOSE

- `securityContext` ↔ `--user`, `--cap-drop`, `--read-only`
- Pod Security ↔ no Docker equivalent

POD SECURITY STANDARDS

LEVEL	ALLOWS
`privileged`	Anything (system add-ons)
`baseline`	Blocks known escalations
`restricted`	Hardened best practice

NAMESPACE LABEL MODES

POD-SECURITY…/	EFFECT
`enforce`	Reject pod
`audit`	Audit log entry
`warn`	kubectl warning
`enforce-version`	Pin rules, e.g. v1.3x

RESTRICTED BLOCKS

FIELD	RULE
`privileged / host*`	Not allowed
`hostPath volumes`	Not allowed
`runAsNonRoot`	Must be true
`capabilities`	Drop ALL

⚠ COMMON MISTAKES

- Enforcing `restricted` on `kube-system` — breaks CNI and CSI pods
- `readOnlyRootFilesystem` without a writable `/tmp` — app crashes
- Images built to run as root — `runAsNonRoot` blocks them
- Assuming PSA blocks everything bad — it only checks pod fields

✓ TIPS & BEST PRACTICES

- Roll out with `warn` + `audit` first, then `enforce`
- `kubectl label --dry-run=server ns shop pod-security…=restricted` lists violators
- Set `USER 10001` in your Dockerfile
- `kubectl debug` profiles help keep debug pods compliant

QUICK SUMMARY

- `securityContext` — user, caps, seccomp
- `privileged / baseline / restricted`
- `pod-security.kubernetes.io/enforce`

Debugging & Troubleshooting

A DEBUGGING ROUTINE

Work outside in: `get` shows **what** is wrong (status, restarts), `describe` and **events** show **why** the cluster couldn't do it, `logs` show why the app failed, and `exec`/`debug` let you look inside. Most problems are a typo in an image name, a missing ConfigMap key, too little memory, or a selector that matches nothing.

NOTES

- Events expire after about an hour; check them early
- `kubectl debug` adds a tool container to a running pod
- Distroless images have no shell; use `debug`, not `exec`
- `kubectl get pod x -o yaml` → `status.containerStatuses` has the exact reason

SECURITY CHECK

- `port-forward` to prod databases bypasses network rules
- Audit and time-limit it

DEBUG TOOLKIT

```
kubectl get pods -o wide                          # STATUS, RESTARTS, NODE
kubectl describe pod api-7d9f-x2k                 # Events: the why
kubectl get events -n shop --sort-by=.lastTimestamp
kubectl logs api-7d9f-x2k -c api --tail=100 -f    # follow one container
kubectl logs api-7d9f-x2k --previous              # before the last crash
kubectl logs deploy/api --all-containers --since=15m
kubectl exec -it api-7d9f-x2k -- sh               # shell inside
kubectl debug -it api-7d9f-x2k --image=busybox:1.36 --target=api
kubectl debug node/worker-1 -it --image=ubuntu    # host fs at /host
kubectl port-forward pod/api-7d9f-x2k 8080:8080   # reach it locally
kubectl top pods --containers                     # needs metrics-server
kubectl run net --rm -it --image=nicolaka/netshoot -- bash
```

CODE EXPLAINED

- → `--previous` — the crashed container's last output
- → `deploy/api` — logs from one pod of the Deployment
- → `--target=api` — share the app container's process namespace
- → `debug node/` — privileged pod on that node
- → `netshoot` — dig, curl, tcpdump, nc in one image
- → `--since=15m` — or `--since-time`, `--timestamps`
- → `port-forward` — bypasses Services and Ingress to isolate the app

PRODUCTION CHECK

- Events + logs into your observability stack
- `kubectl debug` instead of shells in images

IN DOCKER / COMPOSE

- `logs` / `exec` / `cp` ↔ the same Docker commands
- `port-forward` ↔ `-p` for a quick test

SYMPTOM → CAUSE → CHECK

STATUS	LIKELY CAUSE	CHECK
`Pending`	No room, PVC unbound, taints	`describe pod` Events
`ImagePullBackOff`	Wrong tag, private registry	Image name, `imagePullSecrets`
`CrashLoopBackOff`	App exits on start	`logs --previous`
`OOMKilled`	Over memory limit	Raise limit, check leaks
`CreateContainerConfigError`	Missing ConfigMap/Secret key	`describe pod`
`Running, no traffic`	Not Ready, bad selector	`get endpointslices`

EXIT CODES

CODE	MEANING
`0`	Clean exit
`1`	App error
`126 / 127`	Not executable / not found
`137`	SIGKILL (OOM, grace over)
`143`	SIGTERM
`139`	Segfault (SIGSEGV)

⚠ COMMON MISTAKES

- Reading current logs of a crash-looping pod — use `--previous`
- Restarting pods before reading Events — the evidence disappears
- `exec` into distroless images — no shell; use `debug`
- Blaming Kubernetes for app config errors — read the app's own logs first

✓ TIPS & BEST PRACTICES

- Install `stern` to tail logs across many pods
- `k9s` gives a live terminal UI over all of this
- Check endpoints before blaming the network
- `kubectl get pods --field-selector status.phase=Failed -A` finds casualties

QUICK SUMMARY

- `get → describe → logs → exec/debug`
- `logs --previous` — why it crashed
- `debug` — tools without rebuilding the image
- `events` — cluster's side; `logs` — app's side

Helm & Kustomize

helm repo add · upgrade --install · values.yaml / -f / --set · helm template / rollback · kustomization.yaml · overlays · apply -k

PACKAGING AND VARIANTS

Helm is a package manager: a **chart** is a set of templated manifests plus `values.yaml`; installing it creates a versioned **release** you can upgrade and roll back. **Kustomize** is template-free and built into kubectl: plain YAML **bases** plus per-environment **overlays** that patch them. Many teams use Helm for third-party apps and Kustomize for their own.

NOTES

- Charts come from HTTP repos or OCI registries (`oci://`)
- Release state is stored as Secrets in the release namespace
- `kubectl kustomize` renders without applying
- No server component since Helm 3; Helm 4 keeps the same workflow

SECURITY CHECK

- Third-party charts can install cluster-wide RBAC
- `helm template` and review before installing

HELM

```
helm repo add prom https://prometheus-community.github.io/helm-charts
helm show values prom/kube-prometheus-stack > values.yaml
helm upgrade --install mon prom/kube-prometheus-stack \
  -n monitoring --create-namespace -f values.yaml --set grafana.enabled=true
helm list -A ; helm history mon -n monitoring
helm rollback mon 1 -n monitoring
helm template mon prom/kube-prometheus-stack -f values.yaml   # render only
```

CODE EXPLAINED

- → `upgrade --install` — idempotent install-or-upgrade
- → `-f` then `--set` — later values win
- → `template` — render YAML locally; diff or review it
- → `images` / `replicas` — common edits without patches
- → `configMapGenerator` — adds a hash suffix, so pods roll on change
- → `helm show values` — start your values file from the defaults
- → `resources: [../../base]` — overlays reference a shared base

KUSTOMIZE: OVERLAYS/PROD/KUSTOMIZATION.YAML

```
resources: [../../base]            # base: deployment.yaml, service.yaml
namespace: shop-prod
images: [{ name: ghcr.io/acme/api, newTag: "1.4.2" }]
replicas: [{ name: api, count: 5 }]
patches: [{ path: memory-patch.yaml }]
configMapGenerator: [{ name: api-cfg, literals: [LOG_LEVEL=warn] }]
# kubectl apply -k overlays/prod
```

PRODUCTION CHECK

- Pin chart versions; `helm diff` before upgrades
- Values per env in git

IN DOCKER / COMPOSE

- Helm / Kustomize ↔ Compose files + override files
- `helm rollback` ↔ redeploy the old Compose file

HELM VS KUSTOMIZE

	HELM	KUSTOMIZE
Style	Go templates + values	Patches on plain YAML
Install	Separate CLI	`kubectl -k`
Rollback	`helm rollback`	Via Git
Best for	Third-party apps	Your own apps

CHART LAYOUT

PATH	HOLDS
`Chart.yaml`	Name, version
`values.yaml`	Defaults
`templates/`	Manifests
`charts/`	Dependencies
`values.schema.json`	Optional validation

WHERE NEXT

TOPIC	TOOLS
GitOps	Argo CD, Flux
Operators	CRDs + controllers
Policy	Kyverno, Gatekeeper
Mesh	Istio, Linkerd
Packaging	Helmfile, Timoni

⚠ COMMON MISTAKES

- `helm install` in CI — fails on rerun; use `upgrade --install`
- Long `--set` chains — unreviewable; keep values files in Git
- Mixing `helm` and `kubectl edit` on the same release
- Editing rendered chart output by hand — next upgrade overwrites it

✓ TIPS & BEST PRACTICES

- Pin chart versions with `--version`
- `helm diff` plugin previews upgrades
- Kustomize can inflate Helm charts via `helmCharts:`
- `--rollback-on-failure` (Helm 4) / `--atomic` (Helm 3) auto-reverts

QUICK SUMMARY

- `helm upgrade --install` — releases
- `kustomization.yaml` — base + overlays
- `kubectl apply -k` — apply an overlay
- `helm rollback` — previous release revision

Quick Reference — kubectl Commands

contexts & namespaces · get / describe · apply / delete · rollouts & scaling · logs / exec · nodes · auth can-i

CONTEXTS & NAMESPACES

COMMAND	DOES	#
`config get-contexts`	List contexts	03
`config use-context prod`	Switch cluster	03
`config set-context --current --namespace=shop`	Default ns	03
`create ns shop`	New namespace	03
`get pods -n shop · -A`	One ns · all	03
`config view --minify`	Active config	03

GET & INSPECT

COMMAND	DOES	#
`get pods -o wide`	IP + node	04
`get deploy web -o yaml`	Live object	04
`get pods -l app=web -w`	Filter + watch	04
`describe pod web-0`	Details + Events	04
`explain deploy.spec`	Field docs	04
`api-resources`	All kinds	04

CREATE, APPLY & DELETE

COMMAND	DOES	#
`apply -f file.yaml · -f dir/ -R`	Create or update	05
`diff -f file.yaml`	Preview changes	05
`create deploy web --image=… --dry-run=client -o yaml`	Generate YAML	05
`delete -f file.yaml`	Remove objects	05
`apply -k overlays/prod`	Kustomize	23
`edit deploy web`	Live edit (drift)	05

ROLLOUTS & SCALING

COMMAND	DOES	#
`set image deploy/web web=nginx:1.28`	New version	08
`rollout status deploy/web`	Wait for it	08
`rollout undo deploy/web`	Roll back	08
`rollout restart deploy/web`	Bounce pods	08
`scale deploy/web --replicas=5`	Scale	08
`rollout history deploy/web`	Revisions	08

LOGS, EXEC & DEBUG

COMMAND	DOES	#
`logs pod -f · --previous`	Stream · last crash	22
`exec -it pod -- sh`	Shell inside	22
`debug -it pod --image=busybox --target=app`	Ephemeral tools	22
`port-forward svc/web 8080:80`	Local access	12
`get events --sort-by=.lastTimestamp`	Timeline	22
`top pods --containers`	Per-container usage	22

NODES, SCALING & ACCESS

COMMAND	DOES	#
`cordon · drain --ignore-daemonsets · uncordon`	Maintenance	18
`taint nodes n1 gpu=true:NoSchedule`	Repel pods	18
`top pods · top nodes`	Usage	17
`autoscale deploy web --max=10 --cpu-percent=70`	HPA	19
`auth can-i create deploy --as=…`	Check RBAC	20
`get nodes -o wide`	Node IPs, versions	01

CONFIG, SECRETS & STORAGE

COMMAND	DOES	#
`create cm app --from-literal=K=v --from-file=f`	ConfigMap	15
`create secret generic db --from-literal=pw=…`	Secret	15
`get secret db -o jsonpath='{.data.pw}' | base64 -d`	Read a value	15
`get pv,pvc -A`	Storage status	16
`create secret tls t --cert=c --key=k`	TLS Secret	15

HELM

COMMAND	DOES	#
`helm repo add name URL`	Add chart repo	23
`helm upgrade --install rel chart -f values.yaml`	Install/upgrade	23
`helm list -A · history rel`	Releases	23
`helm rollback rel 1`	Roll back	23
`helm template rel chart`	Render only	23

⚠ COMMON MISTAKES

- Running against the wrong context — check before you delete
- `delete ns` deletes everything inside
- `kubectl edit` changes are lost on the next apply

✓ TIPS & BEST PRACTICES

- `alias k=kubectl` + completion
- `-o name | xargs` for bulk work
- `kubectl explain` beats searching docs for field names

QUICK SUMMARY

- All commands are `kubectl …`
- `#` = sheet with details
- Sheet 25 covers YAML and status codes

Sheet 25

Quick Reference — Manifests & Troubleshooting

apiVersion / kind · short names · manifest skeletons · probe & resource snippets · status → cause → fix

APIVERSION / KIND

KIND	APIVERSION	SHORT
Pod · Service · ConfigMap · Secret	v1	po · svc · cm
PersistentVolumeClaim · ServiceAccount	v1	pvc · sa
Deployment · StatefulSet · DaemonSet	apps/v1	deploy · sts · ds
Job · CronJob	batch/v1	— · cj
Ingress · NetworkPolicy	networking.k8s.io/v1	ing · netpol
HorizontalPodAutoscaler	autoscaling/v2	hpa
PodDisruptionBudget	policy/v1	pdb
Role · RoleBinding (Cluster…)	rbac.authorization.k8s.io/v1	—
HTTPRoute · Gateway	gateway.networking.k8s.io/v1	—

SKELETONS & SNIPPETS

NEED	YAML
Any object	`apiVersion · kind · metadata: {name, labels} · spec`
Deployment	`spec: {replicas, selector: {matchLabels}, template: {metadata: {labels}, spec: {containers}}}`
Service	`spec: {selector: {app: web}, ports: [{port: 80, targetPort: http}]}`
Resources	`resources: {requests: {cpu: 100m, memory: 128Mi}, limits: {memory: 256Mi}}`
Readiness	`readinessProbe: {httpGet: {path: /ready, port: http}, periodSeconds: 5}`
Env from Secret	`valueFrom: {secretKeyRef: {name: db, key: pw}}`
Hardening	`securityContext: {runAsNonRoot: true, allowPrivilegeEscalation: false}`

STATUS → LIKELY CAUSE → FIX

STATUS	LIKELY CAUSE	FIX
Pending	No capacity, taint, unbound PVC	`describe pod`; add nodes, toleration, StorageClass
ContainerCreating (stuck)	Volume mount or CNI problem	Events; check PVC and CNI pods
ImagePullBackOff	Bad name/tag, no registry auth	Fix image; add `imagePullSecrets`
CrashLoopBackOff	App exits at start	`logs --previous`; check command, config
OOMKilled	Memory limit too low	Raise limit; profile memory
Evicted	Node out of memory or disk	Set requests; clean disk; `delete pod`
CreateContainerConfigError	Missing ConfigMap/Secret or key	Create it; check key names
Service, no endpoints	Selector mismatch or pods not Ready	`get endpointslices`; fix labels/probe

DEFAULTS WORTH KNOWING

SETTING	DEFAULT
terminationGracePeriodSeconds	30
revisionHistoryLimit	10
progressDeadlineSeconds	600
Job backoffLimit	6
probe period / timeout / failures	10s / 1s / 3
NodePort range	30000–32767

SELECTORS & LABELS

SYNTAX	MEANS
`-l app=web,tier!=db`	AND of conditions
`-l 'env in (prod,stage)'`	Set match
`label po x env=prod --overwrite`	Set label
`label po x env-`	Remove label
`-L env`	Label as column
`--field-selector spec.nodeName=n1`	Field filter

⚠ COMMON MISTAKES

- Selector ≠ pod labels → no endpoints
- `:latest` → unpredictable pulls

✓ TIPS & BEST PRACTICES

- `kubectl explain <kind>.spec` for any field
- Generate skeletons with `--dry-run=client -o yaml`

QUICK SUMMARY

- Events explain most failures
- Details: Sheets 07, 12, 17, 22